DESIGN PROPOSAL

Composable Rust UI for Bevy

A behavior-first specification for declarative composition, responsive layout, and shaped frame regions.

SwiftUI-style composition.
Rust ownership and types.
Bevy-native application integration.

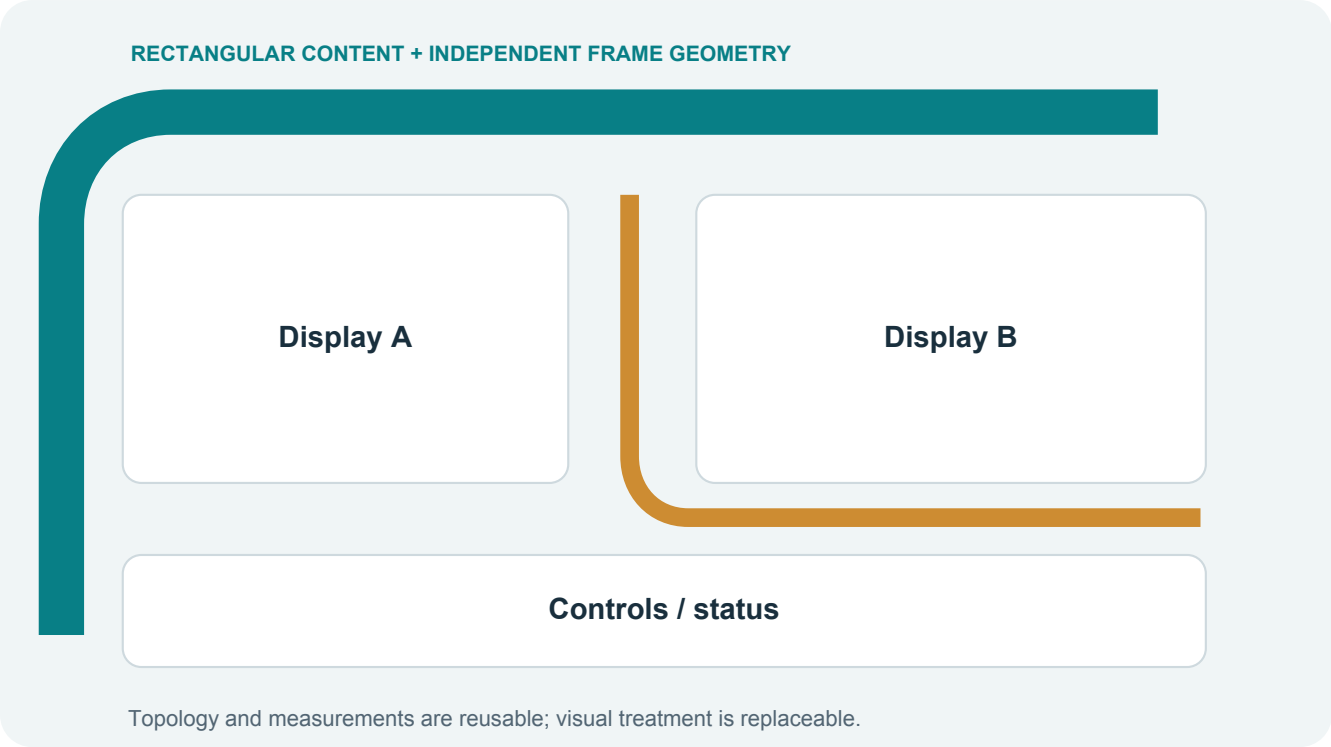


Figure 01. Content slots and shared frame regions are separate parts of the same interface. The colors are diagram notation, not a proposed product theme.

The central proposal is a retained UI runtime behind small Rust view values. It combines predictable box layout with geometry that can describe hairline borders, thick curved rails, cutouts, and shared boundaries between displays.

Status: Specification draft 0.1. All library names and Rust APIs in examples are proposed. This document does not claim an implemented or benchmarked library.

Prepared 21 October 2026. Intended audience: UI framework authors, Bevy application engineers, and designers defining reusable interface systems.

01 / INTENT

Design goals and decisions

Build interfaces as reusable values, with ordinary controls and unusual frames using the same composition model. LCARS-like displays are a demanding geometry use case; the framework must also support restrained panels, forms, dashboards, and game overlays without prescribing their appearance.

Requirement	Proposed contract
Composition	Small views, reusable components, ordered modifiers, keyed collections, typed environment, and local state.
Layout	Stacks, wrapping flex layout, two-dimensional grids, overlays, scrolling, and an explicit custom layout protocol.
Frame geometry	Ordinary per-panel borders plus shared rails and contour regions with curves, gaps, junctions, and independent content insets.
Runtime	Retained identity, incremental invalidation, deterministic input ownership, accessibility semantics, and a Bevy adapter.

The decisions that hold the system together

D1. A view description is disposable; identity and state persist in the runtime. **D2.** One layout runtime owns all logical rectangles. **D3.** Shared borders have one explicit owner. **D4.** Region geometry is resolved after layout and supplies painting, clipping, and hit testing. **D5.** Controls expose behavior separately from their visual style.

Scope and reading map

Pages	Subject	Pages	Subject
3-5	Composition, identity, state	6-9	Layout and responsive behavior
10-13	Borders and shared regions	14-16	Style, input, accessibility
17-19	Bevy integration and example	20-23	Conformance, delivery, references

Version 1 is a screen-space UI system for desktop and native mobile Bevy apps. It includes reusable controls and shaped frames, not a browser engine, DOM, CSS parser, native platform widget wrapper, or general CAD constraint solver. World-space surfaces and arbitrary text flowing around curves are future extensions.

02 / MODEL

Three structures, one interface

The composition tree describes intent. The runtime expands it into a layout tree, a frame graph, and a semantic tree. These structures share stable node identities but need not have the same shape. A decorative region has geometry without a control role; a composite button can expose one semantic node while painting several items.

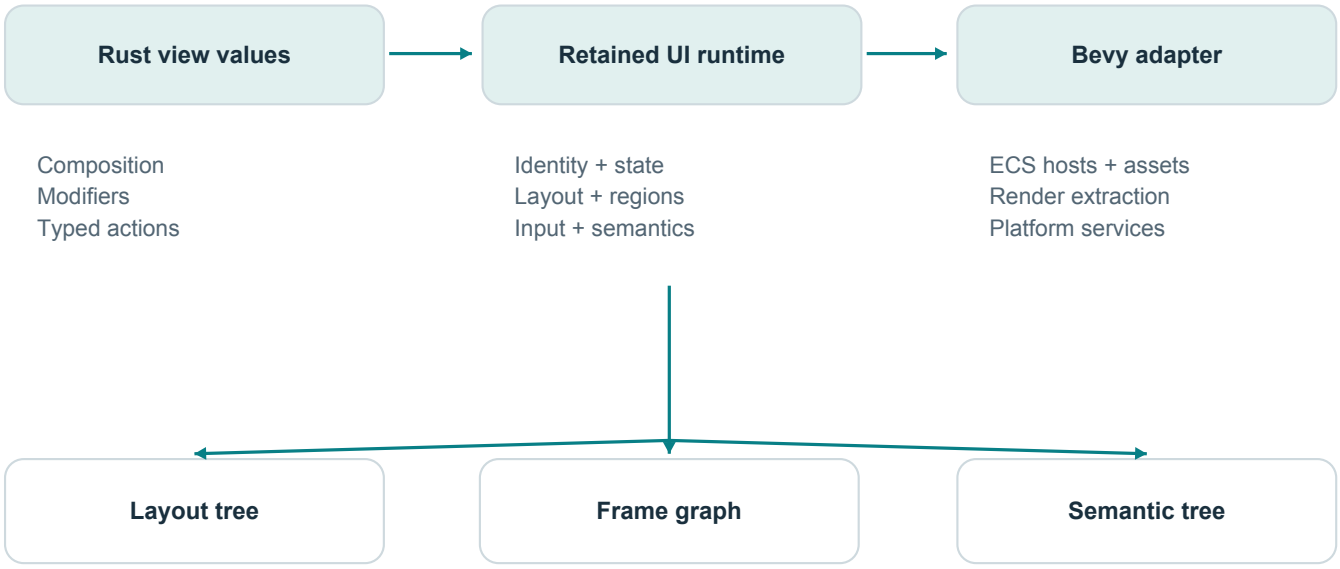


Figure 02. View values are lowered into a persistent runtime. Bevy is the application and rendering host; it is not the public authoring model.

Term	Meaning in this specification
View	A short-lived Rust value describing a UI contribution.
Component	A reusable composition function or type that produces views; distinct from a Bevy ECS component.
UI node	A retained identity with properties, subscriptions, optional state, and links to host entities.
Slot / region	A rectangular content allocation / a filled geometric area. A region may have holes.
Rail / frame graph	A band around a routed centerline / named rails, junctions, gaps, and content slots owned together.
Snapshot	An immutable committed set of layout, geometry, paint order, hit regions, and semantics.

A UI node may own zero, one, or several Bevy entities. Entity IDs are implementation details, never public keys. A mounted root targets one window, camera, or render texture and receives a typed platform environment. Multiple roots have isolated focus, state, and scale.

03 / AUTHORING

A Rust composition language

Use ordinary Rust constructors, tuples, closures, and methods. The baseline requires no procedural macro; an optional markup macro may lower into exactly the same constructors. Static generics provide composition, while explicit type erasure supports runtime-selected subtrees.

```
pub trait View: Sized {
    fn build(self, cx: &mut BuildCx) -> NodeId;
}

fn status_panel(model: &Status) -> impl View {
    VStack::new((
        Text::new("Telemetry").role(TextRole::Heading),
        HStack::new((
            Text::new(model.summary.clone()),
            Spacer::new(),
            Button::new("Refresh", Action::Refresh),
        )).gap(dp(8)),
    ))
    .gap(dp(12))
    .padding(Insets::all(dp(16)))
    .border(Border::uniform(dp(1), Token::Outline))
}
```

Authoring contract. Constructors are inert. Building reads a frozen model snapshot, creates descriptions, and registers dependencies. It cannot mutate the Bevy World, perform I/O, or run actions. Primitive views implement `View`; reusable components return `impl View`. Async work runs through keyed effects after commit.



Text.padding(12).background(fill) paints the padded bounds.

Figure 03. Modifiers wrap from left to right. Reversing padding and background changes the painted bounds.

Required containers are HStack, VStack, ZStack, Flex, Grid, Scroll, VirtualList, FrameLayout, and CustomLayout. Composition helpers are Empty, Either, Optional, Keyed, and ForEach. AnyView erases a subtree type but retains its explicit identity. State belongs to the inner node, so changing only paint modifiers does not reset it.

04 / LIFETIME

Identity, state, and application data

Identity and lifetime draw on the SwiftUI model [2]. Each node identity is a root-scoped path. Static children use stable structural slots; dynamic siblings require application keys. A key is unique within its sibling scope and includes a node-kind discriminator. Rebuilding the same identity updates properties and handlers in place.

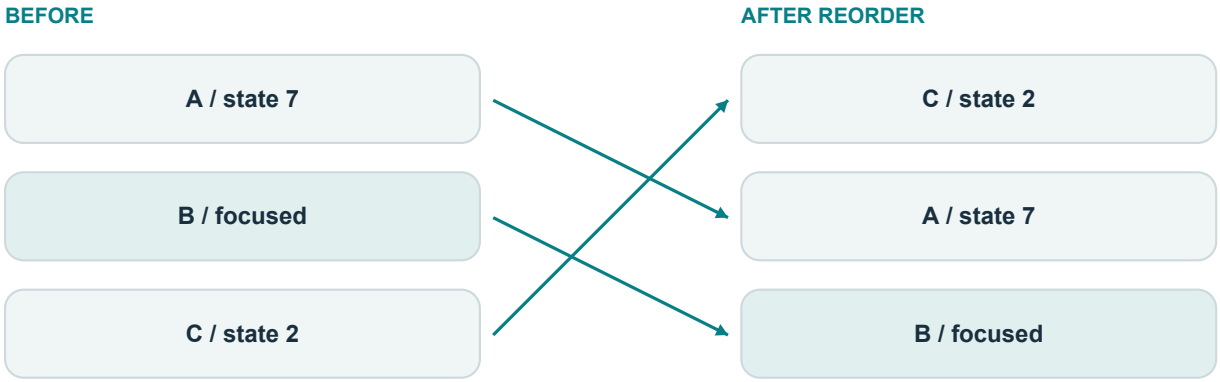


Figure 04. Keyed nodes retain state and focus through reordering. Collection indices are not valid keys when items can move.

Situation	Required behavior
Same key + compatible kind	Retain local state, focus, subscriptions, scroll offset, and associated host entities.
Changed key or incompatible kind	Unmount old node, cancel effects and pointer capture, then initialize a new node.
Duplicate sibling keys	Reject the subtree update atomically; keep the last valid subtree, or an inert error placeholder on first mount. Report both source locations.
Removed then reinserted node	State resets unless the application explicitly stores it outside the subtree.

State has a single owner

`cx.state(StateKey::Expanded, || false)` returns a typed StateHandle keyed by node and field, not by call order. Reads register dependencies; queued updates apply at the next reduction boundary. A Binding pairs a getter with a typed write action. App data remains in Bevy resources or components and is read through explicit selectors over a frozen snapshot.

Handlers own Send + Sync + 'static data or action IDs; they cannot retain World borrows. Bevy systems reduce actions in a declared order and publish revisions. Replaced handlers supersede earlier captures. Async results carry an owner generation; results for an unmounted or replaced owner are discarded.

05 / LAYOUT VOCABULARY

Broad layout without a CSS engine

All layout uses logical device-independent pixels (dp), with origin at top-left, x rightward and y downward. Rectangles use f32 logical coordinates until device mapping. Directional edges use start/end and resolve through the environment. Version 1 supports horizontal writing with LTR and RTL inline direction.

Primitive	Properties and behavior
HStack / VStack	Single line; gap, alignment and baseline; intrinsic basis, grow 0, shrink 1 by default. Spacer uses basis 0 and grow 1. No implicit SwiftUI layout priority.
Flex	Row/column, wrap, basis, grow, shrink, min/max, main/cross alignment, and separate row/column gaps. No margin collapse.
Grid	Fixed, auto, min-content, max-content, fr and minmax tracks; row/column spans, named rectangular areas, and row-major non-dense auto placement.
ZStack / overlay	ZStack sizes to the union of in-flow children. An overlay receives the primary bounds and does not enlarge the primary layout.
Frame / positioning	Explicit min/ideal/max sizes and alignment. Absolute children leave normal flow; offsets and transforms change paint/hit placement, not sibling allocation.
Scroll / VirtualList	Finite viewport, explicit scroll axes, measured content extent. VirtualList mounts visible keyed items plus overscan and retains estimates.
FrameLayout / custom	FrameLayout reserves named corridors around slots. CustomLayout implements measurement and placement with the same child interface.

Typed values and intentional limits

Length values include dp, percent, viewport-relative units, em/rem, auto, and intrinsic sizing keywords. fr is a grid track unit, not a generic length. A full percentage value is written pct(100). Each property accepts only its applicable value types. Negative padding, border widths, gaps, and radii are invalid; offsets and explicit margins may be negative.

The layout subset draws on Flexbox and Grid [4, 5]. Browser compatibility is not promised. Subgrid, floats, fragmentation, vertical writing, sticky positioning, and selector-based styling are outside version 1. Unsupported values produce diagnostics rather than silently approximating another algorithm.

06 / LAYOUT PROTOCOL

Measure, place, then resolve geometry

SwiftUI offers a useful compositional idea: parents propose sizes and children return measured sizes [1]. This design makes hard constraints explicit alongside a proposal. Flex and Grid use Taffy through a runtime adapter; custom layouts use the same measurement service [6].

```
struct Proposal { width: AxisOffer, height: AxisOffer }
enum AxisOffer { Unspecified, MinContent,
                 MaxContent, Definite(f32) }
struct Limits { min: Vec2, max: Vec2 } // max may be +inf
struct Measured { size: Vec2, baseline: Option<f32> }

trait Layout: Send + Sync + 'static {
    fn measure(&self, cx: &mut MeasureCx,
        offer: Proposal, limits: Limits) -> Measured;
    fn place(&self, cx: &mut PlaceCx, bounds: Rect);
}
```

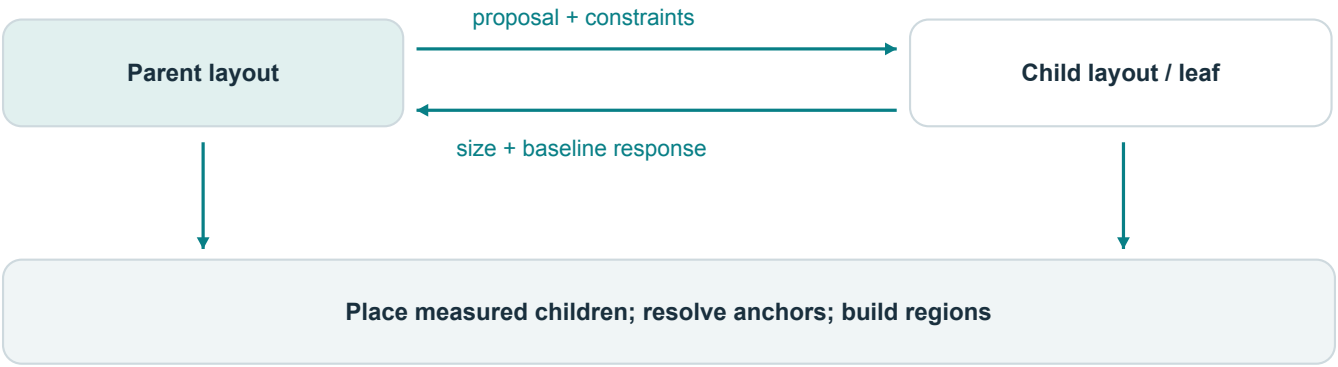


Figure 05. Measurement may probe a child multiple times. Placement has no state changes or back edges into already placed ancestors.

L1. Every measurement returns finite, non-negative size within Limits. A definite proposal is a preference unless a dimension is fixed by equal limits. Overflow content does not enlarge a hard-constrained box. Baselines are optional offsets from the border-box top edge.

L2. Place receives final bounds and may remeasure descendants under definite offers. MeasureCx exposes ordered child IDs and child.measure(offer, limits). PlaceCx assigns each in-flow child exactly one local rectangle. Results are memoized by node revision, offer, limits, font/environment revision and scale-sensitive inputs. Layout caches cannot contain state changes.

L3. Layout is acyclic. Measurement never reads descendant placement or resolved frame geometry. Geometry observers enqueue changes for the next transaction; they cannot cause an unbounded same-frame relayout. Cycles are diagnosed with the participating node path.

07 / SIZING RULES

Deterministic sizing and allocation

Box model and resolution order

Declared width/height use border-box sizing by default. The content box equals the border box minus border insets and padding, clamped to zero. Margin lies outside and never collapses. Resolve definite container dimensions, intrinsic contributions, flex/grid allocation, child remeasurement, alignment, and finally absolute descendants. min greater than max is an error, not an implicit preference.

Percent widths and heights resolve against the corresponding definite containing content dimension. Percentage padding and margins resolve against its definite inline width. An indefinite percentage remains auto during intrinsic measurement; it resolves only if a definite basis is later assigned. Percentage border widths are excluded. A responsive container condition cannot depend on its own final size.

Flex allocation

Use the pinned Taffy Flexbox algorithm for basis, freezing at min/max, and repeated free-space distribution. Positive free space follows grow weights; shrink follows shrink factor multiplied by base size. Defaults explicitly use min width/height zero so text can shrink; use min-content to preserve intrinsic minima. A fixed item uses equal limits and cannot shrink. Gaps are deducted before allocation [4, 6].

Numerical example: A 600 dp row with two 12 dp gaps has 576 dp for items. Bases are 120, 180, 180 dp. The 96 dp surplus with grow weights 0:1:2 yields widths 120, 212, 244 dp. A 230 dp max on the last item freezes it, leaving widths 120, 226, 230 dp.

Grid and placement

Explicit tracks and spans precede row-major auto placement. Auto tracks honor intrinsic contributions; fr divides remaining space after fixed tracks and gaps. minmax(0, fr(1)) permits shrinkage below text min-content. Grid areas must be rectangles; unsupported or overlapping named definitions fail validation. Visual overlap is allowed only through explicit placement. Reading order remains composition order [5].

Rounding and failure

Compute in logical coordinates. Snap shared device-space boundaries once, deriving adjacent widths from those boundaries; do not round every width independently. Transforms apply after layout and must be invertible for hit testing. Non-finite inputs are rejected. Oversized content follows explicit overflow policy: visible, clip, scroll, or ellipsis for supported text.

08 / ADAPTATION

Responsive content and native surfaces

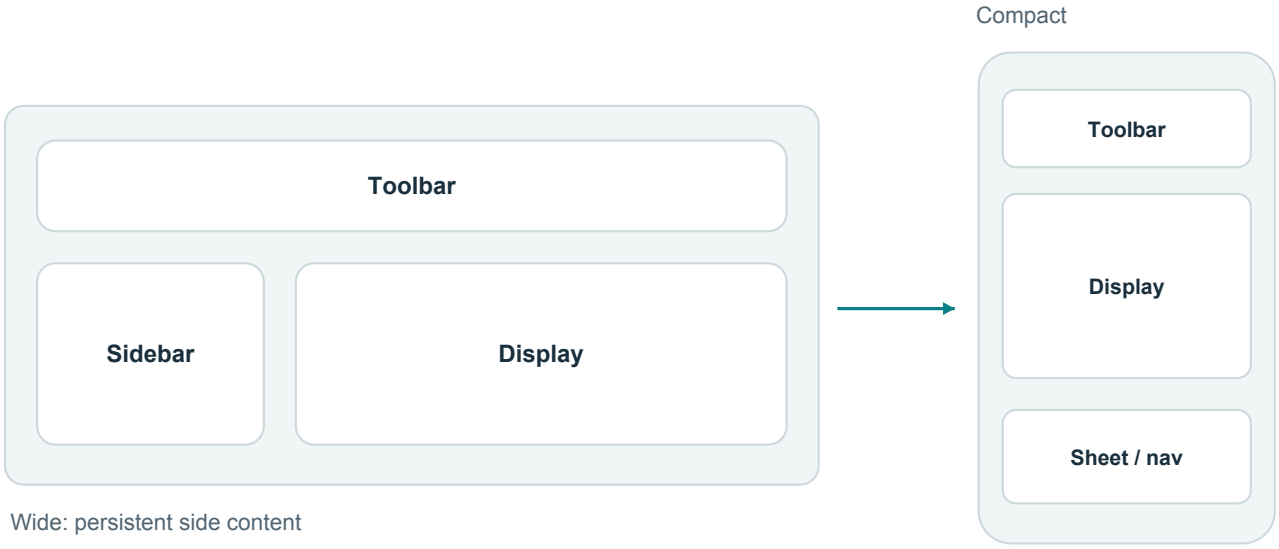


Figure 06. A wide layout becomes a compact arrangement while preserving the content identities and application state.

Available space. A root receives viewport size, platform safe-area insets, UI scale, text scale, direction, input capabilities, and keyboard occlusion. The default content root respects safe areas. Edge-to-edge decoration requires an explicit opt-out; interactive content still uses safe-area and keyboard insets.

Conditions. Adapt chooses a layout plan from root constraints or a definite ancestor query size, before measuring descendants. A compact default can apply below a declared threshold, such as 640 dp. A condition must not read its own measured output. Switching a layout plan reuses named slots; replacing the composition branch follows the identity rules on page 5.

Text. Measurement and drawing must share shaping inputs: font asset/version, fallback chain, language, direction, size, line spacing and wrap width. Cache min/max-content widths and width-dependent heights. Font arrival invalidates affected measurements. Expose baseline alignment, wrapping, max lines, ellipsis, and selectable text. Inline rich text is a text-engine feature, not nested box flow.

Scroll. A scroll viewport is finite on enabled axes; its content may measure unbounded there. Scroll offsets are clamped after extent changes. Nested scrolling passes unconsumed deltas to the nearest ancestor. Virtualized items retain stable keys, estimate unmounted extents, and preserve an anchor item plus local offset when heights change.

Text editing additionally requires grapheme-aware caret movement, selection, composition/IME, clipboard, platform keyboard behavior and semantic text actions. These belong to a tested platform/text adapter. A painted text label must not be presented as a complete editable text control.

09 / BORDERS

Ordinary borders stay ordinary

Border is a box decoration for one panel. It supports per-edge thickness and paint, elliptical per-corner radii, and explicit content padding. It reserves its declared thickness inside the border box. Outline is a separate paint-only effect, useful for focus rings; it contributes no layout insets.

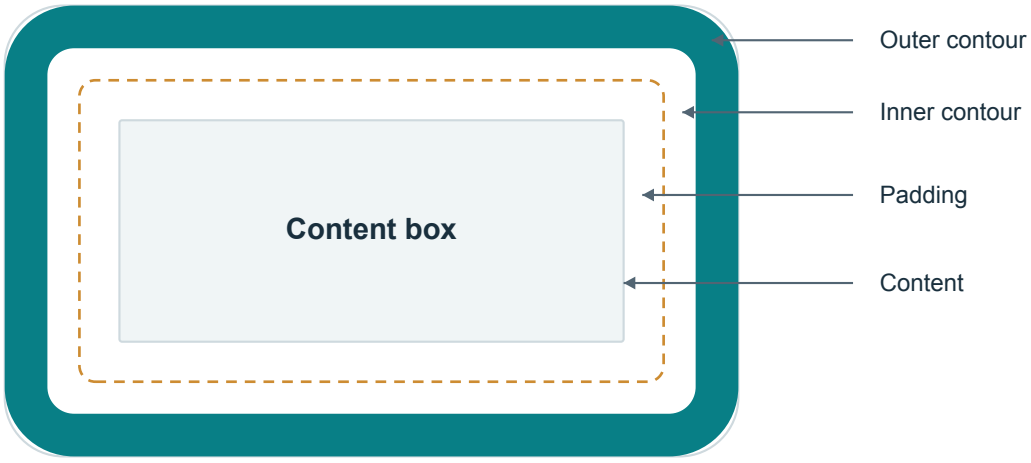


Figure 07. A border occupies the region between outer and inner contours. Padding controls content clearance independently of border thickness.

B1. Radius normalization. For each edge, compare the sum of its two corner radii to the edge length. Multiply all outer radii by the smallest ratio needed to fit, capped at 1. Derive inner radii by subtracting the adjacent horizontal and vertical thicknesses, each clamped to zero. This follows the rounded-box construction in CSS Backgrounds and Borders [3].

B2. Thick borders. Inset the outer box by the four thicknesses to obtain the inner box. If opposite insets consume an axis, that inner dimension becomes zero and no content area remains there. Filled border geometry may occupy the whole outer shape. A warning identifies an empty content box; the runtime must never produce inverted rectangles.

B3. Paint and clips. Backgrounds select border-box, padding-box, or content-box bounds. Rounded border paint does not automatically clip child content. clip_content(InnerContour) is explicit. Per-edge color transitions meet at a deterministic corner bisector; solid fills are required in version 1, with dash patterns optional.

Use this API for cards, buttons, forms, separators, rounded panels and hairline outlines. A shared shaped boundary between sibling panels belongs to a FrameLayout owner, described on pages 11-13.

10 / REGION ALGEBRA

Contours and rails are real regions

A Region is a filled area in local coordinates. Geometry, paint, layout reservation, content placement, clipping, and input participation are independent properties. A transparent cutout is empty space, not a patch painted in the background color.

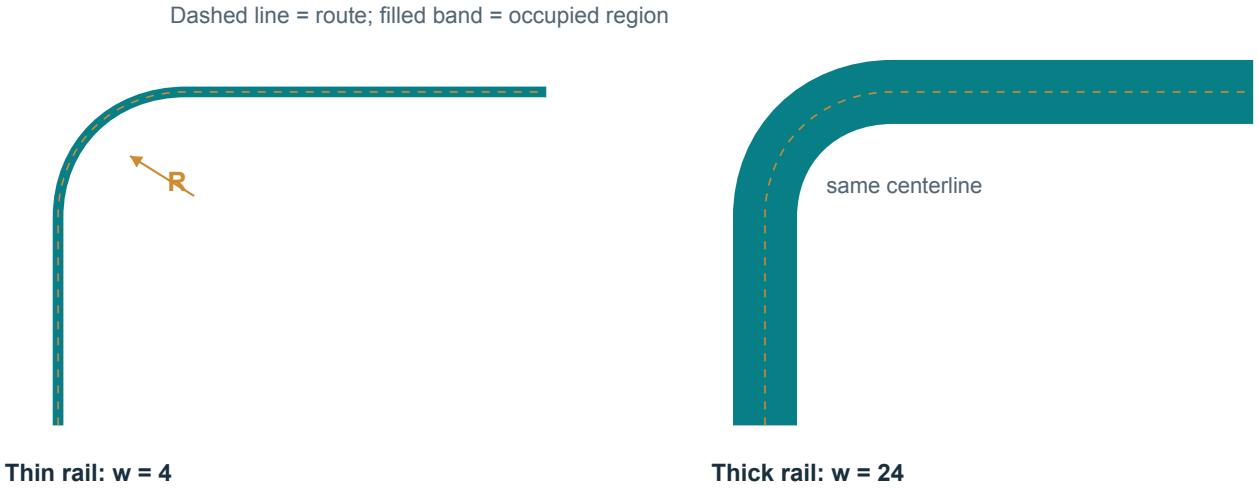


Figure 08. A rail is the occupied band around a route. Width can change without changing the route or the content semantics.

Construct	Contract
Contour region	A closed outer contour minus explicit closed holes. Supports line, quadratic/cubic Bezier, and elliptical arc segments. Even-odd filling determines inclusion.
Rail region	A centerline with non-negative width, cap, join and corner radius. Uniform width per edge is required. Width transitions are explicit junction regions.
Boolean composition	Union, intersection, and difference normalize valid contours into one region. Shared junctions use union to avoid duplicate coverage.
Reserved space	An explicit box inset or a FrameLayout corridor allocation. Measuring a region never feeds a paint-dependent value back into its own container.

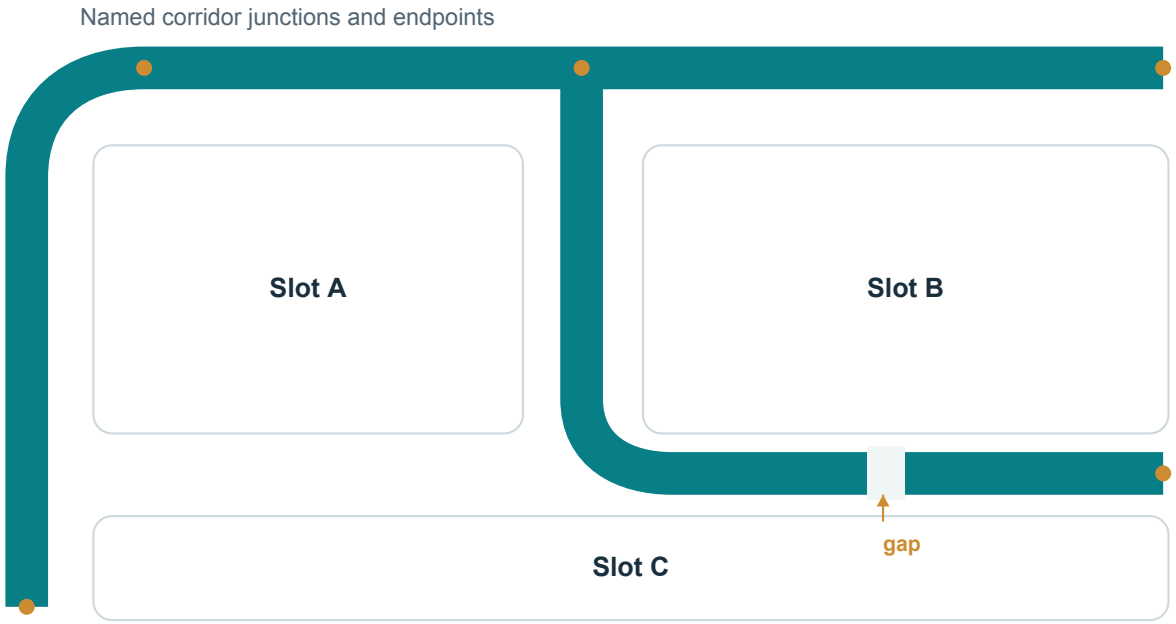
Rail math. On a circular bend with centerline radius R and width w, outer radius = R + w/2 and inner radius = R - w/2. A regular bend requires R at least w/2. Radius R is the centerline radius, unlike Border radii, which describe the outer contour. A filled elbow can instead use a contour with independently defined outer and inner curves.

Thickness is specified in dp. A separate Hairline paint primitive may request one device pixel, but reserves no space; it cannot define a content corridor. Layout uses an explicit logical reservation when clearance is needed.

11 / SHARED FRAMES

A frame graph connects displays

FrameLayout composes rectangular content slots and explicitly reserved rail corridors. A frame graph addresses named corridor endpoints and junctions. This supports a broad curved sidebar, a thin separator, or a continuous band that turns between several displays without making each display draw a competing border.



One frame owner paints the shared boundary once.

Figure 09. Content slots occupy disjoint allocations. A shared rail has one owner; endpoints, junctions, and gaps remain anchored as slots resize.

F1. Allocation before geometry. Reserve corridor widths using track sizes, not solved paint bounds. A corridor must contain the maximum rail half-width, bend extent, and declared clearance. Layout places slots and corridors together; anchors are then resolved into the committed coordinate space. Geometry outside a declared reservation is an error unless the rail is explicitly an overlay.

F2. Explicit routing. Version 1 uses named orthogonal routes with rounded bends. The application declares which corridors a rail traverses; no global obstacle-avoidance router is required. Topology remains stable during resize. Reusing a slot name across responsive plans preserves that slot identity and state.

F3. Gaps and segments. Gaps are excluded arclength intervals along the centerline. They split the rail into independently capped segments. Intervals may use absolute dp or fractions of total length and are normalized, clipped and merged. A cutout label is placed in a separate measured slot; its required clearance is declared before placement.

12 / GEOMETRY CONTRACTS

Curves, joins, and difficult cases

The geometry resolver must produce one normalized region representation per owner. Painting, clipping, and painted-region hit testing use that same representation and the same transform. An approximate shader must stay within the declared coverage tolerance of its CPU representation.

Condition	Required result
Short adjacent route legs	A 90-degree bend consumes R on each adjacent leg. Two bends on one leg consume R1 + R2. Clamp scales requested radii by one route-wide factor so every leg fits. Then apply the minimum-radius rule below; Strict rejects any required adjustment.
No feasible round bend	If available R is less than w/2, default Clamp uses a bevel and emits a geometry warning. Strict returns an error. Neither produces a negative inner radius.
Caps and joins	Support butt, square, round caps; bevel, round and miter joins. Miter limit defaults to 4 times half-width, then falls back to bevel.
Junction / changing width	Union incident edge bands with an explicit junction region. A frame owner composites common paint once. Multi-paint junctions require an explicit partition.
Self-intersection / invalid hole	Reject self-intersecting input contours, touching or crossing holes, NaN, infinity and invalid arc parameters. Overlapping valid inputs may be normalized by Boolean union.
Zero and near-zero sizes	Width zero or area zero paints and hits nothing. Degenerate segments are removed. A completely empty region may still have a layout reservation.
Clip and overflow	Intersect clip ancestors in order. Scissor is only a rectangular fast path; shaped clips require a mask/stencil or equivalent coverage path for every descendant draw item.

Accuracy and lifecycle

Use a target-space flattening tolerance of at most 0.25 device pixels at the supported transform scale. CPU region classification must agree with rendering outside a 0.5 pixel edge band. Antialias fringe is visual only. Transform or DPI changes invalidate scale-dependent tessellation. Cache by path, widths, radii, gaps, fill rule, clip dependencies, and tolerance.

Boolean normalization must be deterministic within documented numeric tolerances. Clip depth and path complexity have declared limits; the baseline target is at least 16 nested shaped clips and 4,096 input segments per region. Exceeding a limit returns an actionable diagnostic and keeps the last valid subtree.

13 / APPEARANCE

Style is a replaceable policy

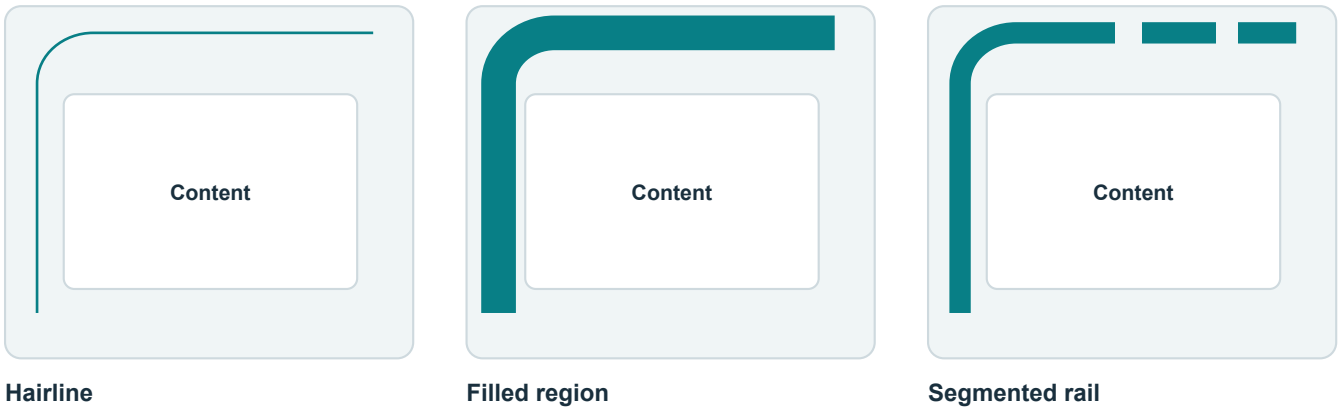


Figure 10. The same content slots and frame route accept several border treatments. The examples illustrate capabilities, not a theme to adopt.

Styles consume semantic roles and interaction state. A Button owns focus, activation and disabled behavior; a ButtonStyle decides what to draw. A FrameStyle supplies paints and geometry parameters for named rails. Tokens represent meaning such as Surface, Outline, Accent and Danger, rather than embedding a particular palette in layout.

Predictable precedence

Resolve each property from: framework default, inherited environment, named style, explicit view modifier. Within an explicit modifier layer, later writes win for the same property; wrapping modifiers remain ordered nodes. Component styles receive one resolved state set and choose a branch explicitly, avoiding implicit selector specificity. Paint tokens may be inherited; layout properties do not inherit unless documented.

Environment values include theme, density, text scale, direction, locale, reduced motion, contrast preference and input modality. A component reads only what it uses. Changing a paint token invalidates paint subscribers; changing a spacing or font token also invalidates measurement.

Deterministic paint order

Within a stacking context, sort by explicit z-index and then composition order. A local context is atomic in its parent; descendants cannot escape it. Paint background and border before content, overlays after children, and focus outlines last. Root layers order scene overlay, normal UI, modal and tooltip content. Portals change paint placement but retain logical event ancestry and focus ownership.

Transitions with stable identity

Animate opacity, transform, paint, radii, rail width and compatible contour parameters. Interpolate numeric properties only between valid geometries. Incompatible path topology crossfades snapshots; it must not interpolate unrelated vertices. Layout animation is explicit and incurs remeasurement each sample; transform-only motion does not.

All animated presentation geometry, including clipping and hit testing, is committed together. Exiting nodes are non-interactive and leave the semantic tree immediately; their paint may remain until the transition ends. Reduced motion disables spatial transitions by default. Geometry changes that exceed declared corridor reservations require layout or an explicit overlay.

14 / INTERACTION

Input belongs to an explicit owner

Snapshot N is the authority for paint and input.

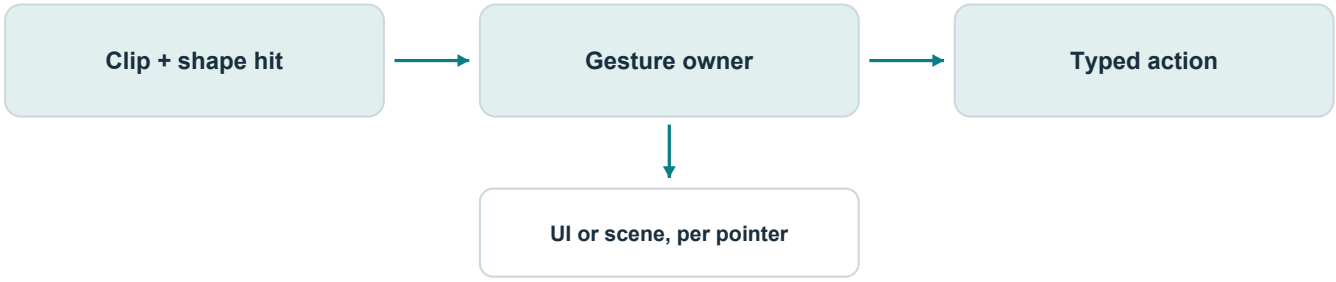


Figure 11. Shape-aware hit testing precedes gesture ownership. A UI gesture must not simultaneously manipulate the scene behind it.

Every event is evaluated against one committed snapshot. Hit testing traverses reverse paint order, applies ancestor clips, inverts transforms, and then tests the node input shape. Decorative regions default to pass-through. Controls default to their allocated box; a rail becomes interactive only with an explicit role, handler, and chosen hit shape.

Policy	Behavior
HitShape	Box, content contour, painted region, or explicit inflated target. An inflated target is intersected with the permitted input clip.
Dispatch	Capture phase from root to target, target phase, then bubble to root. stop_propagation stops traversal; prevent_default suppresses a control default action.
Activation	Pointer down inside an enabled control captures that pointer. Release inside activates once; cancellation or release outside does not. Keyboard and accessibility actions use the same command.
Gesture arbitration	Tap, drag, scroll and pinch compete in an explicit arena. Platform-scaled thresholds and priority resolve the winner. Lost recognizers receive cancellation.
Scene routing	A pointer beginning in a blocking UI input region belongs to the UI until release/cancel. A pointer beginning in a pass-through region may belong to the scene. Ownership is per pointer; a multi-touch recognizer may combine only pointers owned by the same surface.

Focus is root-scoped. Tab follows semantic composition order unless an explicit order is supplied; directional navigation uses resolved bounds and an explicit override graph. Modal roots trap focus, route Escape/Back to dismissal, and restore focus to the invoker. Focused scroll descendants are brought into view.

15 / USABILITY

Semantic controls and platform behavior

A visual style must not determine whether a control is usable. Each interactive node exposes role, accessible name, value, enabled/selected/expanded state, supported actions, and resolved bounds. Decorative rails and cutouts are absent from the semantic tree. Meaningful groups and headings remain in reading order even when Grid changes their visual positions.

Required primitive	Behavior contract
Button / toggle / choice	One activation path; pressed, hover, focus-visible and disabled states. Toggle reports current value; choice groups enforce declared single or multiple selection.
Slider / numeric control	Bounded value, step, keyboard adjustment and semantic increment/decrement. Drag capture survives the pointer leaving the visual track.
TextInput	Selection, grapheme editing, IME, clipboard, validation state, caret visibility and accessible text actions. Password semantics do not expose plain text.
Scroll / list	Keyboard scrolling, focus reveal, stable item keys and virtualized position/count semantics. Semantic navigation can request mounting a distant item.
Dialog / menu	Managed focus, dismissal, modal scope, keyboard traversal and restored focus. Outside clicks follow an explicit dismissal policy.

Targets, contrast, and assistive technology

The default touch profile uses a minimum 44 by 44 dp target, an adjustable product policy rather than a claim that dp equals an Apple point or CSS pixel. Closely spaced expanded targets must be disambiguated by explicit allocation; hidden overlap cannot arbitrarily steal input. Focus indication remains visible around any chosen geometry.

Provide high-contrast tokens and never encode state using color alone. Automated audits can use WCAG 2.2 contrast criteria as a reference: 4.5:1 for normal text, 3:1 for large text, and 3:1 for applicable non-text indicators [10]. These checks do not by themselves establish application conformance.

Export semantics and actions through a Bevy/AccessKit adapter [9], then validate on actual platform assistive technology. Native iOS support requires proof of VoiceOver actions, safe-area changes, keyboard occlusion, touch cancellation, and app suspend/resume; engine primitives alone do not establish that coverage.

16 / BEVY BACKEND

One authority for logical layout

Use a separate layout runtime over the retained node store, with Taffy algorithms for standard containers and custom callbacks for other layouts. The runtime owns logical rectangles. Bevy hosts receive exact resolved bounds, assets, transforms and draw data. Bevy must not independently reflow those same descendants from intrinsic or flex properties.

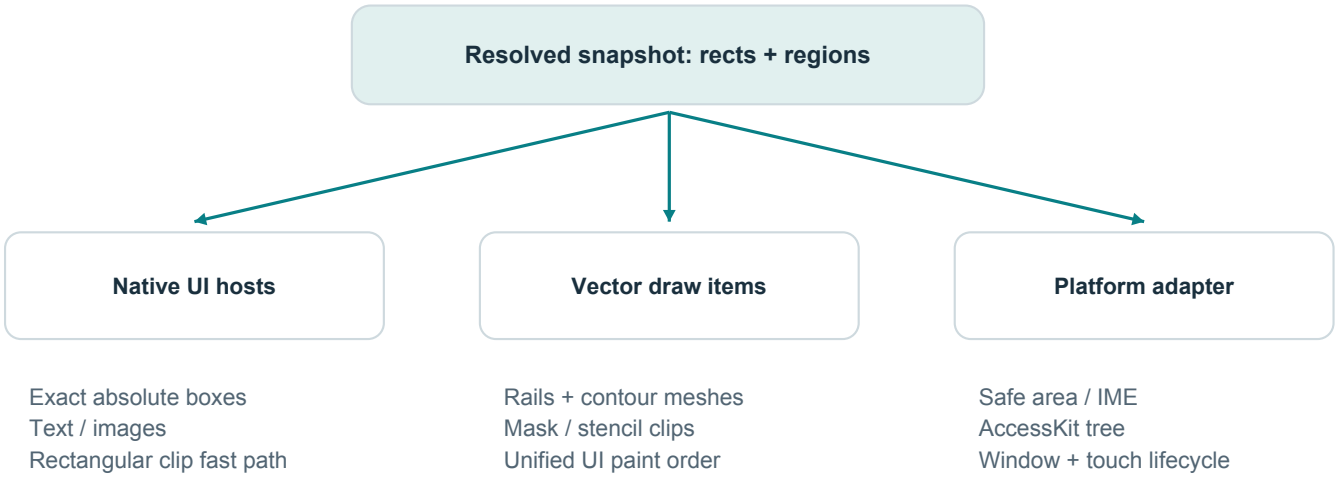


Figure 12. A Bevy adapter presents resolved results. Native UI hosts are a fast path; custom vector and clip rendering is a required extension for general regions.

Host strategy. Emit absolute, explicitly sized border-box hosts with adapter-owned margin, border and padding set to zero; content insets have already been resolved. Native hosts may be flattened under a viewport root. Preserve logical ancestry separately for clips, events and semantics. Inspect Bevy-computed host bounds to verify they match the runtime snapshot, rather than writing private computed fields.

Rendering. Reuse Bevy text, images, simple boxes, and UiMaterial for suitable analytic shapes [7, 8]. General contours and junction unions use tessellated vector draw items in the same UI paint-order domain. A UiMaterial on a parent is not a descendant clip. Arbitrary shaped clipping requires a dedicated clip-aware extraction/render path or offscreen coverage composition.

Capability boundaries. The backend must preserve interleaving of native and vector items, alpha composition and nested clipping. This is a release gate, not an assumed drop-in mesh API. Disable native focus/picking dispatch for adapter-managed hosts so input is handled exactly once. Native interop subtrees declare one measurement boundary; external systems cannot write adapter-owned layout components.

17 / EXECUTION

Transactions, schedules, and invalidation

Frame N uses the last committed geometry for incoming events.



Commit N+1 atomically: geometry, paint order, clips, hit regions, semantics.

Figure 13. Actions change state before composition. The next snapshot is committed as a coherent unit, preventing visual and input geometry from drifting apart.

Frame contract. Route raw input against snapshot N before scene handlers run, reduce typed actions, publish model revisions, compose dirty owners, resolve styles, measure/place, resolve frame regions, and prepare N+1. Reconcile host entities and flush deferred commands before dependent Bevy UI work. Commit the next input snapshot only after host finalization and geometry validation; incoming events retain the snapshot generation they used.

Schedule adapter. Bevy 0.20 exposes UI preparation/content/layout/post-layout/stack sets, with input focus work in PreUpdate and UI layout in PostUpdate [8]. Map runtime work to explicit before/after dependencies in a versioned plugin. A native-host backend must stage final text measurement and absolute box updates before its consuming layout systems. The exact same-frame text measurement hook is a feasibility gate; do not depend on private UiSurface internals.

Change	Minimum invalidation
Action / selected model revision	Subscribed composition owners only; diff their properties and keyed children.
Paint / opacity token	Affected draw data; no layout or path rebuild unless geometry also changes.
Text / font / text scale	Text shaping, intrinsic measure, ancestor layout dependencies, and affected region anchors.
Viewport / corridor / spacing	Affected layout subtree, anchored geometry, clips, hits and semantic bounds.
Path / thickness / gap	Region normalization and mesh/mask; layout only when declared reservations change.

The adapter owns entity lifecycle and a property-level ownership table. An explicit EntitySlot escape hatch hosts application content with fixed measurement and clipping rules. Jobs may compute immutable geometry off-thread, but commits validate node generations and must not expose partially updated trees.

18 / WORKED COMPOSITION

One screen, interchangeable frame profiles

This proposed API assembles two content displays with a shared left-and-top rail and a divider elbow. The plan reserves corridors before route generation. Profile values can produce a fine border treatment or a broad curved band without changing content views, application data, or control behavior.

```
fn console(model: &Console, profile: FrameProfile) -> impl View {
    let plan = ConsolePlan::wide()
        .columns([fr(2), fr(3)])
        .corridor("outer", profile.outer_reserve())
        .corridor("divider", profile.divider_reserve());

    FrameLayout::new(plan)
        .slot("primary", telemetry(&model.primary))
        .slot("secondary", diagnostics(&model.secondary))
        .frame_graph(|g| {
            g.rail("shell", Route::through([
                anchor("outer.bottom"),
                anchor("outer.elbow"),
                anchor("outer.top_end"),
            ]))
            .width(profile.shell_width)
            .bend_radius(profile.bend_radius)
            .paint(Token::Outline);

            g.rail("split", Route::named("divider.elbow"))
                .width(profile.divider_width)
                .gap(Gap::fraction(0.60, 0.66))
                .paint(Token::Accent);
        })
        .adapt_below(dp(640), ConsolePlan::compact())
        .safe_area(SafeArea::Content)
}
```

Example parameter	Fine profile	Broad profile
Shell / divider width	1 / 1 dp	24 / 8 dp
Centerline bend radius	18 dp	40 dp
Outer / divider reservation	24 / 24 dp	64 / 56 dp

ConsolePlan is a reusable application layout component. It defines the named slots, orthogonal corridors, bend envelopes and anchor locations in each plan; these names are not built-in magic. The compact plan stacks the same slots and supplies equivalent routes. Reservations are validated against actual bend bounds and clearance, not accepted solely because they exceed rail width.

19 / CONFORMANCE

Observable acceptance criteria

An implementation conforms to version 1 only when its baseline features pass deterministic logic tests, rendered comparisons, and platform interaction tests. The following fixtures are requirements, not reports of tests performed on an existing implementation.

ID	Fixture and expected result
C01	Keyed reorder A/B/C to C/A/B preserves state, focus and scroll by key. Duplicate keys preserve the last valid subtree and report both locations.
C02	Flex example on page 8 returns 120/212/244 dp; with last-item max 230 it returns 120/226/230 dp. Constraints, gaps and min-size defaults are held fixed.
C03	A 640 dp grid with 16 dp gap and 200 dp fixed first track leaves 424 dp to a minmax(0, 1fr) second track. At 639 dp the declared compact plan is selected.
C04	Border box 300 x 180 dp, widths L24/T12/R8/B6 gives inner box 268 x 162. Uniform outer radii 40 give top-left inner radii 16 x 28 dp.
C05	A circular rail with R40/w24 has radii 52 and 28 dp. R8/w24 with no feasible larger bend bevels with a warning under Clamp; Strict fails.
C06	Two rounded bends on a short leg satisfy R1 + R2 at most leg length. Zero-width rails paint/hit nothing. Invalid self-intersections fail atomically.
C07	A shared T junction at 50% alpha has the same coverage as its arms, with no double-dark overlap. Holes reveal live scene content and do not hit.
C08	Nested shaped clips affect text, images, meshes and hit regions identically. CPU/GPU agreement holds outside the 0.5 pixel edge band at 1x, 1.5x, 2x and 3x scale.
C09	Pointer down on a UI control then drag/release over the scene never orbits the camera. Scene gestures originating through a pass-through hole remain scene-owned.
C10	Rapid theme and viewport changes preserve identity. Color-only updates cause zero layout solves and zero path tessellations in an instrumented unchanged tree.
C11	Keyboard, screen reader and touch activate one typed command. Compact/wide resize, text scale 2x, RTL, keyboard occlusion and suspend/resume preserve usable focus.

Render references use pinned fonts, fixed scale, deterministic time and explicit backend settings. Numeric rectangle tolerance is 0.01 dp before pixel mapping. Coverage comparisons allow the defined antialias edge band. Device-level tests remain necessary for text shaping, IME and accessibility.

20 / DELIVERY

Implementation plan and release gates

Deliver the system in vertical slices, validating the hard integration seams early. Keep the public view/layout/geometry contracts independent of Bevy component names so engine upgrades affect the adapter rather than every application view.

Stage	Deliverable and exit condition
A / feasibility	Prove exact-size native hosts, same-frame text measurement, native/vector paint interleaving, nested shaped clips, and shape-aware picking in a small Bevy demo. Stop API stabilization until these agree.
B / core composition	Keyed reconciliation, explicit state keys, model selectors, effects, typed actions, diagnostics, and composition inspector. Pass C01 and input lifecycle fixtures.
C / layout	Stacks, Flex, Grid, overlays, custom layout, responsive slot plans, scroll and text. Pass sizing, intrinsic-measure and resize fixtures with pinned fonts.
D / frame geometry	Border, rail, contour, Boolean union, corridor validation, gaps, junctions, clip paths, and region debugging. Pass geometry and alpha-seam fixtures.
E / platform readiness	Styled controls, editable text, semantics, virtualization, touch, keyboard and native mobile integration. Pass C09-C11 on supported devices.

Performance targets to validate

For a release build, target 60 fps on the project's native iPhone and iPad baseline devices. As an initial engineering budget, aim for combined UI CPU work below 2 ms p95 and UI GPU work below 2 ms p95 for 1,000 mounted nodes, 200 visible text labels, 24 curved rails and 4 nested shaped clips. These are proposed budgets, not measured results. Record hardware, resolution, backend and thermal state.

Benchmark steady idle, one-label updates, theme changes, resize, scrolling, and animated rail width separately. An unchanged tree must perform no reconciliation, layout or tessellation work; rendering may still run. VirtualList mounts the viewport plus bounded overscan rather than the full collection. Track node/mesh/mask memory, cache hit rate, draw calls and offscreen pixels.

Risks and unresolved choices

The main risks are custom clip cost on mobile, consistent text measurement across native hosts, and Bevy render API churn. Gate them in stage A. Choose the vector Boolean/tessellation library and mask-versus-stencil strategy through measured prototypes. Select exact supported OS and graphics backends only after the platform matrix passes. World-space UI and full CSS compatibility remain separate proposals.

21 / SOURCES

Technical references and provenance

The behavior contracts in this document are proposed design decisions. The sources below establish the existing concepts and integration surfaces used to inform them. Bracketed reference numbers in the text point here. Links were checked on 21 October 2026; documentation URLs using latest can change.

[1] Apple - ProposedViewSize

Parent proposals and child measurement; inspiration for the protocol on page 7.

https://developer.apple.com/documentation/swiftui/proposedviewsize

[2] Apple - Demystify SwiftUI, WWDC21

Identity, lifetime and dependencies inform the retained model; the Rust rules are explicitly specified here.

https://developer.apple.com/videos/play/wwdc2021/10022/

[3] W3C - CSS Backgrounds and Borders Level 3

Rounded box radius normalization and inner border geometry.

https://www.w3.org/TR/css-backgrounds-3/

[4] W3C - CSS Flexible Box Layout Level 1

Basis, growth, shrinkage, wrapping and alignment vocabulary.

https://www.w3.org/TR/css-flexbox-1/

[5] W3C - CSS Grid Layout Level 1

Tracks, spans, placement and intrinsic sizing vocabulary.

https://www.w3.org/TR/css-grid-1/

[6] Taffy 0.14.0 - crate architecture

Public low-level layout functions and tree traits permit embedding a custom retained node store.

https://docs.rs/taffy/0.14.0/taffy/

[7] Bevy - UI Material example

Custom shader paint through a MaterialNode; does not establish arbitrary descendant clipping.

https://bevy.org/examples/ui-user-interface/ui-material/

No external artwork is used. All diagrams are original vector schematics. Rust examples illustrate proposed APIs and have not been compiled against an implementation.

22 / COMPATIBILITY

Bevy baseline and design boundaries

[8a] Bevy - Node

Public box, flex and grid properties. The researched documentation identified Bevy 0.20.0.

https://docs.rs/bevy/latest/bevy/ui/struct.Node.html

[8b] Bevy - UiSystems

Public input and UI layout system sets used to define a versioned schedule adapter.

https://docs.rs/bevy/latest/bevy/ui/enum.UiSystems.html

[8c] Bevy - UiMaterial

Shader/material extension surface for native UI nodes.

https://docs.rs/bevy/latest/bevy/ui_render/ui_material/trait.UiMaterial.html

[9] Bevy - accessibility primitives

AccessKit integration primitives and dependency-version coordination; a complete control layer still has to be implemented and tested.

https://docs.rs/bevy/latest/bevy/a11y/index.html

[10] W3C - WCAG 2.2

Contrast, focus and input guidance used as product-quality references, without claiming certification.

https://www.w3.org/TR/WCAG22/

Repository context and assumptions

The workspace package apps/galaxy-menu/Cargo.toml requests Bevy 0.20.0. Locally available Bevy UI sources at 0.20.0 and Taffy 0.14.0 were inspected alongside the public references. An implementation must pin its resolved adapter dependencies and publish the supported Bevy matrix. Taffy internals and Bevy private layout state are not public dependencies of the authoring API.

The existing native iPhone/iPad decision requires safe areas, touch input ownership and a measured 60 fps target. The galaxy backdrop decision permits controls to overlay the scene. This design accommodates both through screen-space roots and explicit input routing; it does not change the current application or choose a new appearance.

Adoption boundary: This is a PDF design proposal. No library, application code, repository ADR, GitHub issue, or benchmark result was created or modified as part of the specification.